RUST · METHOD · WORKED EXAMPLE

Writing a Rust Library with AI

A method, and the ext4 filesystem it was worked out on

Three days of confident wrong answers, then one day to a formatter that matches `mke2fs` field for field — plus a checker, a differ, and a userspace file writer that needs no kernel, no mount and no root.

Glenn West · August 2026

What this covers

The loop

Three days trying to fix code that already existed

The playbook

Five moves — and what the AI could and could not do

The library

What came out of it, feature by feature

The proof

Seven kinds of verification, and what each one alone missed

Every number here comes from the repositories: commit timestamps, test counts, and the output of `e2fsck` and `debugfs` from e2fsprogs 1.47.3.

WHERE IT STARTED

A filesystem that mounted and refused every write

An existing storage engine had a formatter. It produced filesystems that the Linux kernel mounted and wrote to happily. RouterOS — running lwext4 — mounted them too, and then rejected every single write.

> **The trap in that sentence:** "mounts read-write" and "is writable" are different claims. Only the second one is a test. For three days the first one was being treated as evidence.

The formatter was described as proven. It had produced working filesystems for months. That description was the problem — it made the formatter the last place anyone looked.

THE FAILURE MODE

Three days, six confident answers

DAY 1

ROUTEROS

Mounted. Read-write. Healthy.

...and every write refused.

+2 h

CLAUDE

It's the journal. It goes dirty and lwext4 won't replay it. Turn it off.

Journal off. **Still refused.**

+5 h

CLAUDE

Then it's `metadata_csum`. lwext4's support is partial.

Feature dropped. **Still refused.**

DAY 2

CLAUDE

This is a MikroTik bug. lwext4 is an incomplete implementation — we can't fix it from our side.

Blame assigned. **Nothing tested.**

DAY 3

CLAUDE

The formatter is proven — it's produced working filesystems for months. Look elsewhere.

The one place left unexamined.

STOP

GLENN

We have wasted 3 days using "proven". Need a stepback.

The loop breaks from **outside**.

THE FAILURE MODE

Why that loop holds

Every panel is a plausible hypothesis, argued well, and acted on. None of them was tested against a reference — they were tested against the symptom, which stayed the same either way.

What an AI adds to it

An unlimited supply of confident, well-argued next hypotheses. It will not tire, will not get bored, and will not spontaneously say "I have been wrong five times, so my sixth answer is probably wrong too." Two of those panels blamed a *third party's* code — the least falsifiable answer available, and the most comfortable one.

What "proven" did

It made the formatter the one component nobody examined. Proven is a claim about the past — it had produced filesystems the Linux kernel accepted for months, and every one of those months was on 512-byte-sector storage. The claim was true and irrelevant.

> **Nothing in the loop could have found the answer**, because every step took the existing formatter as given and varied something else.

THE TURN

Breaking out

> *"We have wasted 3 days using 'proven'. Need a stepback."*

> *"No need to analyze for the 100000th time. Doing that for days. Nothing worked. Write from scratch from source."*

Two instructions, and they are the whole method:

✔ **Stop adjusting.** Do not extend the existing code. Do not start from it. Do not read it for inspiration.

✔ **Go to the source.** Not the symptom, not the spec alone — the actual implementation everybody else's tools agree with.

Rewriting sounds like the expensive option. It was finished the same day.

Five moves

Everything after this slide is one of these five, worked out on a filesystem. They are in order, and the order matters.

1 · Refuse the loop	Stop adjusting existing code. The decision to define rather than adjust is the whole difference — and it has to come from you.
2 · Write the definition	A handful of constraints that each rule something *out*. Before any code.
3 · Give it one rule	Something that settles arguments by sending them somewhere checkable, rather than inviting another opinion.
4 · Build the differ first	A tool that answers "am I the same as the reference?" — so the answer is a program's, not the model's.
5 · Verify with what isn't you	Your code, your tests and your assumptions fail together. Somebody else's implementation fails separately.

FOCUS, PART ONE

Define the thing before writing it

Four constraints, written down before any code. Each one rules something out, which is what makes it useful.

Constraint	What it rules out
Async, parallel across devices	Reads and writes take `&self`, so one format fans out across block groups and many formats run at once. The original was serial.
Pure Rust, no C	No binding to e2fsprogs, no `unsafe`, no casting a `repr(C)` struct over a buffer. Every field encoded by hand, little-endian.
Device-agnostic	A `BlockDevice` trait is the only seam. No file, no loopback, no round trip through the kernel.
Byte-exactness is testable	No "looks right". Output is compared against recorded real `mke2fs` output for the same geometry.

> **Nothing in this crate reads or writes a path outside the device it was given.** A boundary stated once, worth more than a code review later.

One rule that settles arguments

> *Match what `mke2fs` actually writes, field for field, derived from the e2fsprogs source.*

Where a value is a judgement call in our code and a computed default in `mke2fs`, we compute it the way `mke2fs` does. Not "a reasonable value". Not "what the spec permits". The same value.

Why a rule, not a goal

A goal ("make it correct") invites a new opinion every time something is ambiguous — and an AI has an unlimited supply of plausible opinions. A rule ends the conversation: go and look at what the reference does.

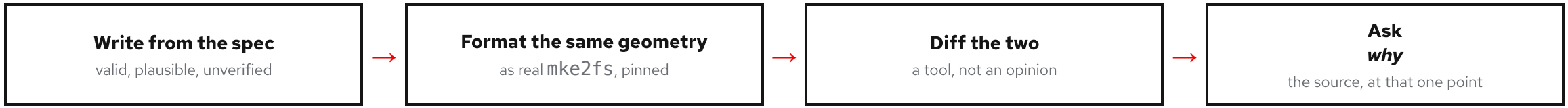
What it buys downstream

A consumer that disagrees with real `mke2fs` output is now a consumer bug, with a reference to point at — instead of an open-ended guess about feature flags. That is exactly the argument that burned three days.

FOCUS, PART THREE

Write from spec. Then find out where you're wrong.

The code was written from the ext4 on-disk specification — not transcribed from anybody's C. The specification is enough to produce a filesystem that is valid in every particular and identical to nobody's. So that is only step one:



> **The last step is the one that does the work.** A difference tells you *that* you disagree; it never tells you who is right. Going to the reference implementation at exactly that field turns a mismatch into a reason — and a reason is what tells you whether to change your code or write down why you did not.

Spot references, not a reading of the whole. You do not need to know how `mke2fs` works. You need to know why it put a 4 there.

FOCUS, PART THREE

What the spec could not answer

Four differences the compare tool found, and what the source said when asked about that one field.

The difference	Why, once the source was asked
Reserved GDT blocks: we wrote zeros	They are not spare space. They *are* the resize inode's indirect blocks — so zeroing them is corruption, and the spec never says so
`flex_bg` metadata in the wrong place at 256 MiB	Placement is an allocation, not arithmetic. A contiguous run collides with group 1's superblock backup
Inode and block sizes differing by volume size	`mke2fs.conf`'s size classes — a configuration file, not a format rule
Journal `csum_v3`: we set it, `mke2fs` did not	Nothing was wrong. `mke2fs` simply does not set it at format time — a difference to record, not to fix

> **That last row is why the step exists.** Three of these were our bugs and one was not, and the diff alone cannot tell them apart.

What the AI was good at — and what it never did

Good at

- Eighteen thousand lines of exacting, tedious, byte-level encoding — offsets, little-endian fields, checksums — without getting bored or sloppy
- Holding a whole on-disk format in view at once, and writing the tests beside the code
- Reproducing a twenty-year-old hash bit-for-bit from a description of it
- Building the comparison tool that then caught its own mistakes

Never did

- ✘ Notice it was looping. Six hypotheses over three days, each delivered with the confidence of the first
- ✘ Distrust its own word "proven"
- ✘ Resist blaming a third party — twice, and wrongly
- ✘ Register that its tests agreeing with each other proved nothing
- ✘ Ask "has anyone measured this on a 4 KiB-sector drive?" — that question came from a person

Focusing it: what actually worked

Instead of	Say
"Make it correct"	**"Match the reference, field for field"** — a goal invites an opinion; a rule sends the question somewhere checkable
"Fix the write path"	**"Write it from the spec. Do not read the old code"** — otherwise the old assumptions come along
"Is it right?"	**"What does the diff say?"** — ask for the difference, not the verdict
"Fix that difference"	**"Why is it there?"** — demand the reason first; three of ours were bugs and one was not
"Do the tests pass?"	**"What would prove you wrong?"** — "the tests pass" is not an answer when you wrote the tests

> **And state boundaries once, in writing.** "Nothing in this crate reads or writes a path outside the device it was given" did more than any review would have.

This is not really about filesystems

The method needs three things: a specification, an implementation everyone already agrees with, and a way to compare your output to its output. Filesystems happen to have all three lying around. So do most things worth reimplementing.

Building	The reference	The differ
A filesystem	`mke2fs`	Format the same geometry, diff the images
A protocol	The canonical server or client	Replay a capture; diff the bytes on the wire
A codec or format writer	The reference encoder	Encode the same input; diff the output
A parser	The implementation everyone else uses	Feed both a corpus; diff the parse trees

> **No reference at all?** Then building the oracle *is* the first task — a second independent implementation, a property test, a model, or a recorded corpus of real-world artefacts. If you cannot say what would prove you wrong, you are already in the loop and have not noticed.

THE RESULT

Two crates

mkfs-ext4

Creates the filesystem and checks it.

- ext2, ext3 and ext4 from one code path — as `mke2fs` does
- Formatter, parallel across block groups
- `fsck`: six passes, check *and* repair
- Structural compare between two filesystems
- CLI: `mkfs-ext4`, `fsck-ext4`

fio-ext4

Fills it in. No kernel, no mount, no root.

- Files, directories, reads and writes at an offset
- Permissions, ownership, timestamps
- Symlinks, hard links, device nodes, FIFOs
- Extended attributes — SELinux labels and ACLs
- tar in and out, streaming; OCI layers

> **It works on a Mac.** None of it needs a Linux kernel to produce a Linux filesystem — which is the point for CI, for unprivileged containers, and for a microcontroller that has no operating system at all.

FEATURES

On-disk feature parity with `mke2fs`

metadata_csum · 64bit · extents · flex_bg · sparse_super · resize_inode · dir_index · filetype · huge_file · dir_nlink · extra_isize · has_journal · meta_bg · orphan_file · mmp · ext_attr

Scale

1 KiB to 4 KiB blocks. 16 MiB to 256 TiB. `meta_bg` past ~200 TiB, where the group descriptor table stops fitting. A 1 TiB sparse filesystem formats in 7 seconds and allocates 353 MiB.

Sector size

Drives report 512 or 4096, and it is a *floor*: a block smaller than a sector cannot be written a block at a time. Our choice matches `mke2fs` in every cell of the capacity × sector-size matrix.

MMP – multiple mount protection

The fence that stops two hosts mounting one volume read-write and destroying it. Stamp a sequence, wait twice the check interval, re-read, refuse if it moved – and `mmp_nodename` makes the refusal name the holder rather than guessing.

FEATURES

Attributes and ownership are not decoration

A container will not start without the right modes. A tarball cannot be unpacked without owners. An image whose every file is `0644 root:root` is not a root filesystem — it is a directory of files that happens to be shaped like one.

```
vol.write_with("/etc/shadow", data, &Attrs::mode(0o600)).await?;
vol.write_with("/usr/bin/su",  elf,  &Attrs::mode(0o4755)).await?;   // setuid
vol.mkdir_with("/tmp",               &Attrs::mode(0o1777)).await?;   // sticky
vol.chown("/home/gw", 165536, 165536).await?;                        // 32-bit, userns range
vol.mknod("/dev/null", Special::CharDevice { major: 1, minor: 3 },
          &Attrs::mode(0o666)).await?;
vol.symlink("/bin", "usr/bin").await?;
```

Device nodes matter for the same reason: no `/dev/null`, no working image. Creating one normally requires root. Here it is a field in an inode.

Extended attributes — the ones SELinux rides on

Without them, an SELinux system boots into a filesystem it will not let anything run from. ext4 keeps them in two places, and both had to be written:

In the inode

In the spare room after the fixed fields, past `128 + i_extra_isize`. Cheap — no extra block, no extra read.

In a block of its own

When the set does not fit. Reachable only from `i_file_acl`, which is exactly why it is easy to leak.

> **The trap:** block-stored entries need a per-entry hash. The inline form does not. Write a zero hash and the kernel accepts it — `e2fsck` does not: `Extended attribute in inode 25 has a hash (0) which is invalid`.

```
$ getfattr -d -m - /mnt/etc/hostname
security.selinux="system_u:object_r:etc_t:s0"
user.build="fio-ext4"
```

FEATURES

Archives, streamed both ways

The operation an image build actually performs: take a rootfs tarball or a container layer and lay it down with everything intact. Through a kernel that needs root and a mount. Here it needs neither.

```
$ fio-ext4 disk.img untar rootfs.tar
$ skopeo copy ... | fio-ext4 disk.img untar --whiteouts -C /   # a pipe, gzip detected
$ fio-ext4 disk.img tar -z backup.tar.gz
```

Nothing is held in memory

A `Source`/`Sink` pair that needs no runtime, no pinning and no allocation to implement. A file, a pipe, a socket and a registry response are the same thing — and an archive larger than RAM is not a problem.

Layers stack properly

`.wh.name` deletes; `.wh..wh..opq` empties a directory. A replacing file gets a fresh inode, so it inherits none of the mode, owner or labels of the file below it.

FEATURES

Large directories: `dir_index`

Without an index, finding a name means reading the directory until it turns up, and adding one means reading it until a gap turns up. Filling a directory with *n* names costs n^2 block reads. Bearable at a hundred names; not at ten thousand.

> **Grown the way `e2fsck -D` grows one, not the way the kernel does.** The kernel splits one leaf at a time because it cannot stop the world. This builds images — so when a leaf will not take another name, the whole index is rebuilt, sorted by hash and repacked. One code path for the first conversion and every growth after it, instead of a leaf split, a node split and a root promotion that each have to be right alone.

Leaves keep a fifth of each block free, so a rebuild lands about once per two hundred names rather than once per name. The tree is invisible to a linear reader — which is why `dir_index` is a *compatible* feature and an old kernel still mounts the filesystem.

```
7,500 names, 1 KiB and 4 KiB blocks, against a real kernel:
  e2fsck -fn                                  exit 0
  every name resolved through the index, not by listing  0 missing
  a name that was never there                 still absent
  the kernel adding its own name to our tree  ok
  e2fsck -fn, after the kernel wrote          exit 0
```

THE PROOF

Seven kinds of verification

Grey rungs are ours and prove only self-consistency. Red rungs are somebody else's implementation disagreeing with us.

Unit tests	Offsets asserted, not assumed. Necessary. Nowhere near sufficient.
Our own fsck	Six passes over our own output. Shares our assumptions, so it shares our blind spots.
Golden references	7 real `mke2fs` images, byte-comparable — pinned UUID, hash seed and `SOURCE_DATE_EPOCH`.
Structural compare	Our own tool, diffing two filesystems field by field and classifying each difference.
Real e2fsck	e2fsprogs 1.47.3 on Fedora 43, over 11 configurations.
A real kernel	Loop mount, write, unmount, *and check again*. The second check is the one that counts.
An external oracle	`debugfs` computing the same directory hashes independently.

What each one caught that the others didn't

The defect	Caught by	Missed by
crc32c convention. The reference updates a table raw; the Rust crate complements at both ends. Every metadata checksum was wrong.	Real `e2fsck`	All our tests — they agreed with each other
Reserved GDT blocks zeroed. They *are* the resize inode's indirect blocks.	Golden compare	Reading the spec
flex_bg run overlapped group 1's superblock backup at 256 MiB.	Golden compare	Every smaller size
Journal `csum_v3` set at format time; `mke2fs` writes none.	Golden compare	`e2fsck` — it was legal, just not what mke2fs does
1 KiB blocks on a 4 KiB-sector device. The mechanism shipped without the detection.	Measuring a real device	Every test — none had a 4 KiB sector
Zero `e_hash` on block-stored xattrs.	Real `e2fsck`	The kernel, which accepted it
Leaked xattr block on delete. Reachable only from `i_file_acl`.	Real `e2fsck`	Our fsck

THE PROOF

The compare tool, and the list nobody expected

A structural diff between two filesystems, written specifically so "is our output the same as `mke2fs`?" could be answered by a program instead of an opinion. It sorts every difference into three classes:

Identity

UUID, timestamps, label. Expected to differ. Ignore.

Incidental

Free-block counters and the like. Differ without meaning.

Structural

Geometry, features, placement. Any difference here is a bug in one of the two.

> **Not one of these had shown up as a test failure**, because every test we owned checked our output against our own expectations. Replayed today against the formatter as it stood before they were fixed: **33 structural differences over the six references, in four kinds.**

Field	×	ours / mke2fs	What it was
`bg_flags`	16	`0x0000` / `0x0004`	Every ext2 and ext3 group missing `INODE_ZEROED`
`bg_flags`	8	`0x0005` / `0x0007`	ext4 groups missing `BLOCK_UNINIT` — the flag that lets a mount skip untouched groups
`s_lpf_ino`	6	`11` / `0`	We recorded `lost+found`'s inode; `mke2fs` leaves the field zero
`inode 8 i_block`	3	The resize inode — its block map *is* the reserved GDT blocks, which we had zeroed	

Today: zero. The replay above is `examples/replay.rs` — the claim on this slide is a command, not a memory.

THE PROOF

Some things cannot be tested against yourself

A directory hash is the clearest case. Get it wrong and the filesystem is *structurally perfect*: every block valid, every checksum right, every name filed in a leaf. Just the wrong leaf.

> **There is no self-consistent test for this.** Our lookups use our hash, so they agree with themselves no matter what it computes. The only way to know is to ask the implementation everyone else agrees with.

```
$ debugfs -R "dx_hash -h half_md4 hello"
Hash of hello is 0x1746da32 (minor 0x420013b5)
```

26 values, three algorithms, seeded and unseeded, including names long enough to need several transform blocks — pinned into the test suite as constants taken from `debugfs`, not from us. They matched on the first run.

THE PROOF

The bug that passed everything we owned

In an index block, the count-and-limit header and the first index entry *share an address* — deliberately. The first entry has no hash, because those four bytes are the count and limit.
Writing a hash there set the limit to zero. A zero limit puts the checksum tail at the offset of the first entry's block pointer. So the checksum overwrote the pointer.

> **Everything passed.** Unit tests passed. Our own `fsck` passed. Every name was still found — because a pointer that resolved to nothing made the lookup fall back to a linear scan, which worked perfectly and silently.

It only became visible when a directory grew large enough to need a second level of the tree, where there was nothing to fall back to.
Then, and only then: `an index points at a hole`.

THE PROOF

"Did you really fix the 4K?"

Sector-size support had been added, documented and released. The API took a sector size. The tests passed. And on a real 4 KiB-sector device it still produced a 1 KiB-block filesystem — because the *mechanism* had shipped without the *detection*.

1024

our block size, measured on a real 4 KiB loop device

4096

what `mke2fs` chose on the same device

The fix was to ask the kernel — `ioctl_blkszget` — rather than assume 512. And the likely root cause of the three-day bug: a 256 MiB volume on 4 KiB sectors formatted with 1 KiB blocks, which lwext4 will not write to a block at a time.

```
block size chosen, against mke2fs on real devices of each sector size:
  sectors   16M    64M    512M   1G     8G     64G
  512       1024   1024   4096   4096   4096   4096
  4096      4096   4096   4096   4096   4096   4096      ← the size class says 1024;
                                                           the sector overrules it
```

> **No test would have found this.** Not one machine in the test set had a 4 KiB sector. Someone had to ask the question and someone had to go and measure. The matrix above is now a test, and the Linux run re-derives it from a real `mke2fs` so it cannot go stale.

THE CULPRIT

What was actually wrong

Not lwext4. Not MikroTik. A 1,532-line hand-rolled formatter whose defaults were *chosen* rather than derived — and which said so, in its own header comment:

```
//! Feature set is deliberately conservative: EXTENTS|FILETYPE incompat,
//! SPARSE_SUPER|LARGE_FILE|EXTRA_ISIZE ro_compat. No metadata_csum, no
//! 64bit, no bigalloc, no quota, no dir_index. That is the set verified
//! to mount read-write on RouterOS 7.22.2 …

pub const BLOCK_SIZE: u32 = 4096;   ← a constant, not a decision
```

Verified against the symptom

"The set verified to mount read-write" — mounting was the test, and mounting was never the problem. Every feature was picked by whether the symptom moved, so the feature list encoded three days of guesses as settled fact.

Nothing to compare against

Its geometry, its inode counts, its feature list: all invented, all plausible, none of them ever placed beside what a real formatter writes. There was no way to be wrong, because there was nothing to be wrong *against*.

Then it happened again

The hand-rolled formatter was thrown away and replaced with this crate. The templates worked. Weeks later, the same symptom came back — mounts, healthy, every write refused.
Not a bug in the new formatter. **One method left to its default.**

```
// implements BlockDevice for a thin volume
fn size(&self)      -> u64  { … }
async fn read_at(…)         { … }
async fn write_at(…)        { … }
// logical_sector_size() — not implemented, so it answers 512
```

The volume exported 4 KiB sectors. Nothing could ask the kernel, because a thin volume is not a real block device — it has to *say*. It said nothing, so the default spoke for it, and the filesystem got blocks smaller than the sectors underneath them.

> **A block smaller than a sector cannot be written a block at a time.** The kernel hides that behind a read-modify-write. lwext4 does not, and is entitled not to — so it mounts the filesystem, reports it perfectly healthy, and refuses every write.

THE CULPRIT

Where the answer wasn't

Three days produced six confident hypotheses. The answer was in none of the places any of them pointed.

- ✘ Not the journal.
- ✘ Not `metadata_csum`.
- ✘ Not lwext4, and not MikroTik — the implementation everyone was ready to blame was behaving correctly the whole time, and would have been within its rights either way.

It was a default nobody had compared against anything: first a formatter whose feature list was chosen by whether the symptom moved, then a sector size a volume never reported.

> **Both issues are closed.** Not because the reference implementation is cleverer than we are — because `mke2fs` had already met a 4 KiB-sector drive, and we had not.

That is the whole argument for matching a reference field for field. It is not about elegance. It is about inheriting every encounter someone else has already had with the real world.

A journal the filesystem only claimed to have

Every ext4 volume of 1–7 MiB left the formatter with `has_journal` set, zero journal blocks and no journal inode. `mke2fs` never emits that shape. A kernel asked to mount it looks the journal inode up, finds nothing, and refuses.

```
let journal_blocks = match params.journal {
    _ if !compat.contains(HAS_JOURNAL) => 0,      // no feature → no journal
    JournalSize::Default => default_journal_blocks(blocks)
                                .unwrap_or(0),     // no journal → feature stays
};
```

The feature was resolved before the block count was known, and the guard ran one way. Below 2048 blocks the size class answers "no journal" — and nothing cleared the bit. `orphan_file` rode along with it: 128 KiB of a 1 MiB volume, an eighth of the whole thing, on exactly the config, secret and log volumes where nobody looks.

> **Our fsck passed every one of them.** The grey rung, exactly as the ladder drew it: a writer's output verified only by its own reader proves nothing. v2.0.2 clears the feature once the geometry is final — the orphan file goes with the journal, since replaying one is a journal's job — and pass 0 now names the shape: `journal-advertised-but-absent`.

SINCE THIS TALK · EPILOGUE TWO

9.7 MB of file, 5.4 GB of I/O

280×

write amplification, measured per-layer on a real registry build — 1065× at the worst

1.37M

device operations to unpack one 9.7 MB file over NVMe/TCP

0.08ms

per operation — the device was never the problem

Each few KiB of payload re-read and re-wrote the same bitmap, inode-table, extent-node, descriptor and superblock blocks: the device was acting as the only metadata cache. Transport fixes were tried first and *acquitted* the lower layers — coalescing and reordering barely moved it, because interleaved far-offset metadata stops contiguous runs from ever forming.

> **The fix sits at the seam the design left for it.** `CachedDevice` (v2.1.0) wraps any `BlockDevice` in a bounded write-back LRU — sub-block dirty tracking, so a partial write to an uncached block pays no read-modify-write — and fio-ext4 v1.5.0 streams the unpack and allocates from the goal. Found the way the 4 KiB sector was found: not by a test, by a measurement on a real device.

SINCE THIS TALK

Ten days of commits, in five lines

Release	What landed
mkfs-ext4 v2.0.0 Aug 19	A synchronous `no_std` read path — firmware links *this* reader instead of growing a second implementation that drifts against it (#2)
mkfs-ext4 v2.0.2 Aug 23	The phantom journal, previous slide (#3) — plus the fsck check that would have caught it, and the test that proves the check fires
mkfs-ext4 v2.0.4 Aug 23	v2.0.3 shipped an *empty* `Cargo.toml` — a tag pushed with no build between edit and release. The broken tag stands, marked, rather than changing meaning under anyone who fetched it
mkfs-ext4 v2.1.0 Aug 27	`CachedDevice`, the write-amplification fix (#4), with sub-block valid/dirty tracking — and the golden reference images vendored in, which a `.gitignore` had silently kept out
fio-ext4 v1.5.0 Aug 27	Streaming block-by-block unpack, goal-directed allocation, opened through `CachedDevice` (#3 there)

> **Seven issues filed across the two repos; seven closed.** Every one followed the deck's own script: measured on something real, fixed against the reference, pinned by a test that fails without the fix.

EFFORT

What it cost

3

days in the analysis loop, before any of this

1

day from empty directory to a formatter passing e2fsck and kernel mount

20k

lines of Rust across both crates

229

tests, plus 11 configurations against a real kernel

Day one, 06:50 → 18:00	Then
Scaffold, on-disk structures, checksums, golden references, geometry, the formatter, fsck, both CLIs, meta_bg, orphan_file, compare, MMP — and the second crate started at 13:47.	Extended attributes. Tar and OCI layers. `dir_index` maintenance. Twenty-four releases and counting — mkfs-ext4 is at v2.1.0, fio-ext4 at v1.5.0. The long tail is where the interesting bugs were.

The three days produced nothing. The one day produced a filesystem. The difference was not effort or tooling — it was the decision to stop adjusting and start defining.

USE

Where this is worth having

Embedded and firmware

Write files and data onto a removable drive from a device with no operating system — an ESP32 writing an ext4 card that a Linux box then reads. No kernel, no allocator requirement in the archive path, no C toolchain.

Repair where there is no fsck

RouterOS has no `e2fsck`. This one is a library, so the check and the repair can run inside whatever is already there.

Storage engines

Format many thin volumes at once, straight against the volume — no file, no loopback, no round trip. That is what the async and the `&self` signatures were for.

Image and CI builds

Build a root filesystem, with real ownership and SELinux labels, in an unprivileged container or on a Mac. Unpack a container layer without being root.

Building a library with an AI

- ✔ **Break the analysis loop from outside.** An AI will generate hypothesis twenty as confidently as hypothesis one, forever. Noticing you are in the loop is a human job.
- ✔ **Distrust "proven".** It is a description of history, not a test. It made the formatter the last place anyone looked.
- ✔ **Watch for the other implementation's fault.** Two of the six hypotheses blamed MikroTik. It is the least falsifiable answer available and the most comfortable one — and lwext4 had been behaving correctly the entire time.
- ✔ **Write the definition before the code.** Four constraints that each rule something out beat any amount of "make it correct".
- ✔ **Give it one rule that settles arguments.** "Match the reference, field for field" ends ambiguity by sending it somewhere checkable.
- ✔ **Write from the spec, then diff against the reference.** The spec gives a valid filesystem. Only the diff tells you where valid is not the same as compatible — and only the source, asked at that one field, tells you which of you is wrong.
- ✔ **Verify with something that is not you.** Our tests, our fsck and our assumptions all fail together. `e2fsck`, a kernel and `debugfs` fail separately — which is the whole value.
- ✔ **Rewriting is often the cheap option.** Three days adjusting; one day building.

THANK YOU

Writing a Rust Library with AI

MIT OR Apache-2.0 — the MIT arm is GPLv2-compatible, so it imposes nothing on a kernel or RHEL consumer.

```
$ mkfs-ext4 disk.img
$ fio-ext4 disk.img untar rootfs.tar
$ fsck-ext4 -fn disk.img
```

github.com/glennswest/mkfs.ext4.rs **github.com/glennswest/fio.ext4.rs**

The filesystem was never the hard part.

Knowing when to stop analysing it was.